

Configurable Signed-Distance-Field-Aware Planning and Whole-Body Model Predictive Control for Heterogeneous Mobile Manipulators

Xiaochen Miao

Technical University of Munich
Munich, Germany
xiaochen.miao@tum.de

Abstract—Collision avoidance with whole-body model predictive control (WB-MPC) is challenging because mobile manipulators must coordinate base and arm motion under platform-specific kinematic and mobility constraints. This work presents a configurable collision-aware framework that combines Open Motion Planning Library (OMPL) reference planning with WB-MPC. A shared Euclidean signed distance field (ESDF) provides two collision-avoidance layers: OMPL generates a collision-checked path, and WB-MPC maintains clearance while tracking it. On a nonholonomic Stretch and a holonomic mobile UR10, the framework achieves 16.6- and 18.5-mm end-effector reference RMSE with positive executed-state collision margins. The mobile UR10 also requires less OCP solve time, showing the effect of base mobility on optimization difficulty. Deployment on a physical Stretch demonstrates end-to-end operation with real localization, mapping, and command execution.

Index Terms—mobile manipulation, model predictive control, configurable robotics, motion planning, Euclidean signed distance field, ROS 2

I. INTRODUCTION

Mobile manipulators enlarge an arm’s workspace through coordinated base motion, but a controller designed for one platform is often difficult to reuse on another. Robots can have different kinematics, base types, collision models, and command interfaces. Stretch combines a differential-drive base, lift, telescopic arm, and wrist [1]; the mobile UR10 considered here combines a six-axis arm with a holonomic planar base. A reusable framework must support both without changing its core planning and control algorithms.

Global planning and receding-horizon control offer complementary capabilities. A sampling-based planner can find a collision-free route over a large workspace, but its geometric path does not account for tracking error, coupled dynamics, or input limits. WB-MPC handles these effects locally, but its finite horizon is a poor substitute for global routing. The proposed architecture therefore uses OMPL to generate an ESDF-validated reference and WB-MPC to track it while an ESDF collision cost reshapes the predicted trajectory. The shared ESDF keeps both collision-avoidance layers geometrically consistent.

Portability here does not mean using an identical model for every robot. Instead, each URDF/YAML profile is used to generate a robot-specific OCP. The profile determines the controlled joints and task frames, base dynamics, bounds,

collision spheres, horizon, and cost weights. The generated solver therefore changes with the robot, while the planner, MPC construction, and execution code remain shared. At runtime, a platform adapter maps the common state and trajectory interfaces to the corresponding robot driver. The current Stretch adapter additionally performs state fusion, execution preflight, and deadline-based command holds.

Accordingly, this report asks: *Can an ESDF-aware OMPL–WB-MPC pipeline be configured for mobile manipulators with different arm dimensions and base kinematics, while retaining a consistent collision model and explicit hardware interfaces?* The main contributions are:

- a YAML/URDF-parameterized whole-body MPC formulation that selects state and input dimensions, kinematics, bounds, collision geometry, and base dynamics for each robot;
- a two-layer collision-avoidance design in which OMPL performs ESDF-based path validation and WB-MPC applies a differentiable ESDF collision cost along the prediction horizon;
- a cross-platform evaluation on nonholonomic Stretch and holonomic mobile-UR10 models, including tracking, clearance, solver-tail, and runtime-decomposition results; and
- a physical Stretch demonstration integrating Spectacular AI SLAM and an nvblox-derived ESDF with the planning–control loop.

The simulation supports a controlled quantitative comparison of the two robot models. A complementary Stretch real experiment evaluates transfer to physical sensing and actuation.

II. RELATED WORK

A. Mobile-Manipulator Planning and Control Paradigms

Conventional mobile-manipulation pipelines often decompose a task into base navigation followed by arm manipulation. This separation reduces the dimension of each planning problem and permits reuse of established navigation and manipulator controllers, but it discards base–arm redundancy and can produce conservative or suboptimal whole-body motion when the two subsystems must cooperate in constrained environments.

Kinematic whole-body control instead treats the base and arm as one redundant chain and resolves task-space commands

at the velocity level. Seraji extended configuration control to holonomic and nonholonomic mobile manipulators, incorporating base constraints and auxiliary tasks in a unified kinematic formulation [2]. De Luca *et al.* further considered nonholonomic mobile manipulators with steering wheels and used task-Jacobian null-space optimization and feedback linearization to determine otherwise unspecified steering motions [3]. These approaches are computationally efficient, but primarily resolve instantaneous kinematic tasks rather than predicting the coupled system evolution over a finite horizon.

Sampling-based predictive control provides another route to coupled and reactive motion. MPPI updates a nominal control sequence from the costs of sampled trajectory rollouts and can therefore accommodate complex or nonsmooth objectives without differentiating the complete cost. STORM demonstrated GPU-parallel sampling MPC for a 7-DoF manipulator [4], while Pezzato *et al.* extended GPU-parallel rollouts to navigation, contact-rich manipulation, and high-dimensional whole-body control [5]. RAMP combined an MPPI variant with an implicit configuration-space SDF for reactive obstacle avoidance [6]. Such sampling-based methods are flexible, but whole-body collision avoidance requires rollout costs across samples, horizon nodes, and represented robot bodies, commonly motivating substantial GPU parallelism. The present framework instead exploits ESDF gradients within a gradient-based model predictive controller.

B. Whole-Body Model Predictive Control

Whole-body MPC extends coordinated control beyond instantaneous kinematic resolution by optimizing the coupled base–arm evolution over a finite horizon. It repeatedly solves a finite-horizon problem from the latest state and executes only the first portion of the optimized sequence, allowing tracking, effort, smoothness, bounds, and collision terms to be treated together.

Whole-body MPC also couples manipulation with platform stability. Minniti *et al.* formulated manipulation, balancing, and interaction as one finite-horizon optimal-control problem for an inherently unstable ball-balancing mobile manipulator. Their formulation predicts how end-effector tracking and contact-force decisions affect the future balance of the system, and was validated in hardware pose-tracking and door-opening experiments [7]. This work establishes the value of jointly optimizing platform and manipulator behavior, but focuses on dynamic stabilization of a ballbot. The present work applies a unified predictive structure to kinematically different wheeled mobile manipulators. Enabling online whole-body collision avoidance within this structure additionally requires an environmental representation that can be queried efficiently at many prediction states, motivating the distance-field methods discussed next.

C. Distance-Field-Based Collision Avoidance

Conventional collision checking evaluates the geometric relationship between robot links and environmental obstacles explicitly. Depending on the environment representation, this

may involve repeated collision or distance queries between robot meshes or collision primitives and individual obstacle geometries [8]. Such pairwise geometric queries can become computationally expensive when they are repeatedly evaluated for multiple robot links and prediction states within an online optimization loop.

Distance-field representations instead encode obstacle proximity independently of the robot geometry. ESDFs provide continuous clearance values and local spatial gradients, making them suitable for optimization-based motion generation. Voxblox demonstrated incremental ESDF construction from a TSDF for onboard planning [9], while nvblox accelerates dense incremental mapping on a GPU [10].

Distance fields have consequently been integrated into reactive motion-generation systems. Gaertner *et al.* used a dynamically generated ESDF for static collision checking and moving-obstacle models for dynamic scenes within a legged-robot MPC [11]. Chiu *et al.* incorporated an environment SDF into whole-body MPC and demonstrated collision-aware locomotion and manipulation while caching distance and gradient information to reduce environment-query overhead [12]. Both studies were validated on quadrupedal robot platforms: the former focused on legged locomotion, whereas the latter extended collision-aware MPC to legged mobile manipulation. They demonstrate the benefits of distance fields for reactive collision avoidance, but do not directly address a shared whole-body planning and control framework configurable for wheeled mobile manipulators with different arm dimensions and base kinematics.

III. METHOD

A. Robot-Parameterized Control Framework

The framework accepts a URDF and a composed YAML robot profile. The URDF defines the joints and links, kinematic tree, frame transforms, joint types, and arm joint-position limits. The profile selects the controller joint ordering and task frames from that model, and supplies controller-level choices not encoded by the URDF: the base dynamics, base workspace and velocity/input bounds, collision-sphere approximation, horizon, and cost weights. Pinocchio and CasADi combine these inputs to construct a robot-dimensioned symbolic OCP, from which acados generates a robot-specific solver. Changing the arm dimension, base dynamics, or number of collision spheres regenerates the solver but does not change the construction procedure.

At runtime, a control goal is converted into a reference horizon, the ESDF supplies local obstacle information, and the generated solver jointly optimizes base and arm motion. A platform adapter then converts the optimized trajectory into robot commands. Figure 1 summarizes both offline robot specialization and the online feedback-control path.

The following subsections define the dynamics, ESDF collision model, OCP, and task-conditioned reference generation used in this pipeline.

Offline robot specialization

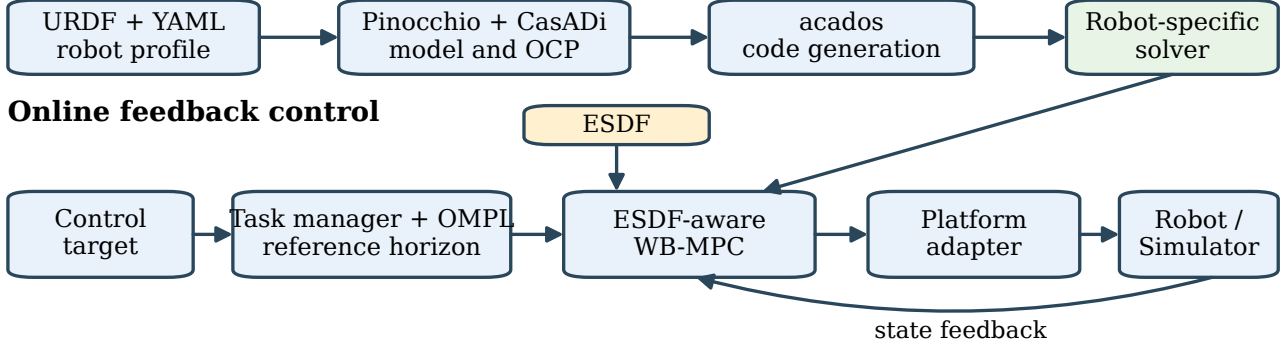


Fig. 1. Robot-parameterized solver generation and online control flow. A URDF and YAML profile specialize the generated solver; control targets and ESDF data then enter the shared WB-MPC path before platform-specific command mapping.

B. Model Predictive Control

1) *Robot-Parameterized Dynamics*: Each robot profile selects the base dynamics branch: an omnidirectional base uses a planar double-integrator model, whereas a nonholonomic base can use the embedded no-slip dynamics derived below. The URDF specializes the arm dimension and kinematics, and all arm joints use acceleration-controlled double-integrator dynamics.

For a robot with n_a independent arm coordinates, the optimizer constructs

$$\mathbf{q} = [x \ y \ \theta \ \mathbf{q}_a^\top]^\top, \quad \mathbf{x} = [\mathbf{q}^\top \ \mathbf{v}^\top]^\top, \quad (1)$$

The corresponding continuous-time state equation is written generically as

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = [\dot{\mathbf{q}}^\top \ \dot{\mathbf{v}}^\top]^\top, \quad (2)$$

where (x, y, θ) is the map-frame planar base pose. Let n_q , n_u , and n_x denote the dimensions of the generalized-coordinate vector, optimizer input, and state vector, respectively. The planar base contributes three coordinates, while the arm contributes n_a independent coordinates; hence, $n_q = 3 + n_a$. Because the optimizer assigns one acceleration input to each generalized coordinate, $n_u = n_q$. The state stacks generalized positions and velocities, giving $n_x = 2n_q$. These dimensions describe the internal optimization model rather than the number of physical actuators. Joint names, bounds, link frames, tool and base links, collision primitives, and URDF are read from the selected robot profile.

For an omnidirectional base, every coordinate follows the common double-integrator model $\dot{\mathbf{q}} = \mathbf{v}$, $\dot{\mathbf{v}} = \mathbf{u}$, and therefore

$$\dot{\mathbf{x}}_{\text{omni}} = [\mathbf{v}^\top \ \mathbf{u}^\top]^\top. \quad (3)$$

For a nonholonomic base in dynamics mode, the same public world-frame state $[v_x, v_y, \omega]$ is retained, but the base part is

replaced by a no-slip model. Its forward and lateral components are

$$v_f = \cos \theta v_x + \sin \theta v_y, \quad (4)$$

$$v_{\text{lat}} = -\sin \theta v_x + \cos \theta v_y = 0. \quad (5)$$

The first two world-frame acceleration variables are similarly projected to $a_f = \cos \theta a_x + \sin \theta a_y$. The embedded base dynamics are

$$\dot{x} = v_f \cos \theta, \quad \dot{y} = v_f \sin \theta, \quad \dot{\theta} = \omega, \quad (6)$$

$$\dot{v}_x = a_f \cos \theta - \omega v_f \sin \theta, \quad (7)$$

$$\dot{v}_y = a_f \sin \theta + \omega v_f \cos \theta, \quad (8)$$

$$\dot{\omega} = \alpha. \quad (9)$$

These equations define the two base-dynamics branches selected by the robot profile.

2) *ESDF Collision Avoidance*: The environment representation is independent of the robot model. Each robot profile instead attaches a set of collision spheres to selected links. Given configuration $\mathbf{q}_k$, forward kinematics determines the center $\mathbf{c}_j(\mathbf{q}_k)$ of sphere j . Define the configuration-dependent clearance as $d_j(\mathbf{q}) \triangleq d(\mathbf{c}_j(\mathbf{q}))$, where the ESDF returns both the signed distance and its spatial gradient.

The exported ESDF is sampled on a regular three-dimensional grid. A query point is first located within a grid cell. The implementation then uses its relative position along the three coordinate axes to trilinearly interpolate both the distance and the stored gradient from the cell's eight corners. The resulting signed distance and spatial gradient are denoted by $\hat{d}(\mathbf{c})$ and $\hat{\mathbf{n}}(\mathbf{c})$, respectively. Under the conservative validity policy used in the experiments, a query is accepted only if it lies inside the grid and all eight interpolation corners are marked valid. The interpolation supplies a continuous local distance-gradient pair to the MPC update.

Because the gridded ESDF is queried outside the generated acados solver, its local values are updated from the warm-start

trajectory before every MPC solve. At shooting node k , the first-order Taylor model around the sphere center $\mathbf{c}_j(\bar{\mathbf{q}}_k)$ is

$$\tilde{d}_{j,k}(\mathbf{q}) = \hat{d}(\mathbf{c}_j(\bar{\mathbf{q}}_k)) + \hat{\mathbf{n}}(\mathbf{c}_j(\bar{\mathbf{q}}_k))^\top (\mathbf{c}_j(\mathbf{q}) - \mathbf{c}_j(\bar{\mathbf{q}}_k)). \quad (10)$$

Thus, the solver retains the robot-specific forward-kinematics expression $\mathbf{c}_j(\mathbf{q})$ but replaces the external map query with a local affine distance-field model. The way this model produces a gradient with respect to the robot coordinates can be seen directly from its differential. Define the point Jacobian $J_{\mathbf{c}_j}(\mathbf{q}) \triangleq \partial \mathbf{c}_j(\mathbf{q}) / \partial \mathbf{q}$. A small configuration change $\delta \mathbf{q}$ moves the sphere center by

$$\delta \mathbf{c}_j = J_{\mathbf{c}_j}(\mathbf{q}) \delta \mathbf{q}. \quad (11)$$

The ESDF distance and gradient evaluated at the warm-start center remain constant during one solve. Differentiating Eq. (10) therefore gives

$$\begin{aligned} \delta \tilde{d}_{j,k} &= \hat{\mathbf{n}}(\mathbf{c}_j(\bar{\mathbf{q}}_k))^\top \delta \mathbf{c}_j \\ &= \hat{\mathbf{n}}(\mathbf{c}_j(\bar{\mathbf{q}}_k))^\top J_{\mathbf{c}_j}(\mathbf{q}) \delta \mathbf{q}. \end{aligned} \quad (12)$$

By the gradient definition $\delta \tilde{d}_{j,k} = (\nabla_{\mathbf{q}} \tilde{d}_{j,k})^\top \delta \mathbf{q}$, comparison of the two expressions yields

$$\nabla_{\mathbf{q}} \tilde{d}_{j,k}(\mathbf{q}) = J_{\mathbf{c}_j}(\mathbf{q})^\top \hat{\mathbf{n}}(\mathbf{c}_j(\bar{\mathbf{q}}_k)). \quad (13)$$

In physical terms, each column of the point Jacobian describes how one robot coordinate moves the sphere center, while its dot product with $\hat{\mathbf{n}}(\mathbf{c}_j(\bar{\mathbf{q}}_k))$ measures how strongly that motion increases or decreases the local obstacle distance. The code does not explicitly construct this matrix product or query a new ESDF gradient inside acados. CasADi obtains it by automatically differentiating the local distance model; the interpolated gradient remains fixed until the next MPC update.

For sphere radius r_j and configured safety distance d_s , the geometric margin at prediction node k is

$$m_{j,k} = d(\mathbf{c}_j(\mathbf{q}_k)) - r_j - d_s. \quad (14)$$

Inside the solver, d is replaced by the Taylor model in Eq. (10), giving $\tilde{m}_{j,k} = \tilde{d}_{j,k}(\mathbf{q}_k) - r_j - d_s$. The collision condition is implemented as a soft cost rather than a hard inequality constraint. Specifically, the implementation uses a smooth relaxation of the hinge violation $\max(0, -m)$ and then squares it:

$$v_\varepsilon(m) = \frac{1}{2} \left(\sqrt{m^2 + \varepsilon^2} - m \right), \quad (15)$$

$$\phi(m) = \frac{1}{2} \mu v_\varepsilon(m)^2 = \frac{\mu}{8} \left(\sqrt{m^2 + \varepsilon^2} - m \right)^2. \quad (16)$$

As $\varepsilon \rightarrow 0$, $\phi(m)$ converges to $\frac{1}{2} \mu \max(0, -m)^2$. The total collision cost over the prediction horizon is then

$$J_{\text{obs}} = \sum_{k=0}^{N-1} \sum_{j=1}^{n_c} \phi(\tilde{m}_{j,k}), \quad (17)$$

where n_c denotes the number of collision spheres attached to the robot. The Stretch and mobile-UR10 profiles supply 34 and

93 link-attached spheres, respectively. At every MPC update, ESDF values and gradients are queried along the warm-start trajectory and supplied to all shooting nodes. The formulation is unchanged when the number or placement of collision primitives changes.

3) *Whole-Body MPC Formulation:* At each cycle the controller solves

$$\min_{\mathbf{x}_{0:N}, \mathbf{u}_{0:N-1}} \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k, \mathbf{r}_k) + \ell_N(\mathbf{x}_N, \mathbf{r}_N) + J_{\text{obs}} \quad (18)$$

$$\begin{aligned} \text{s.t. } \quad & \mathbf{x}_{k+1} = f_d(\mathbf{x}_k, \mathbf{u}_k), \\ & \underline{\mathbf{x}} \leq \mathbf{x}_k \leq \bar{\mathbf{x}}, \quad \underline{\mathbf{u}} \leq \mathbf{u}_k \leq \bar{\mathbf{u}}. \end{aligned} \quad (19)$$

To make the tracking metric explicit, let $\mathbf{e}_k = h(\mathbf{x}_k) - \mathbf{r}_k$ collect the base-pose, end-effector-pose, base-velocity, and end-effector-velocity residuals. Heading error is wrapped on $SO(2)$, and end-effector orientation error is computed with the $SO(3)$ logarithm. The noncollision stage and terminal losses are

$$\begin{aligned} \ell(\mathbf{x}_k, \mathbf{u}_k, \mathbf{r}_k) &= \frac{1}{2} \|\mathbf{e}_k\|_{\mathbf{W}_k}^2 + \frac{1}{2} \|\mathbf{x}_k\|_{\mathbf{Q}_x}^2 \\ &\quad + \frac{1}{2} \|\mathbf{u}_k\|_{\mathbf{R}_u}^2 + \ell_{\text{arm}}(\mathbf{q}_k), \\ \ell_N(\mathbf{x}_N, \mathbf{r}_N) &= \frac{1}{2} \|\mathbf{e}_N\|_{\mathbf{W}_N}^2. \end{aligned} \quad (20)$$

Here $\|z\|_{\mathbf{W}}^2 \triangleq z^\top \mathbf{W} z$ defines the weighted squared Euclidean loss. The matrices $\mathbf{Q}_x$ and $\mathbf{R}_u$ penalize state and control effort, while ℓ_{arm} is the optional arm-extension shaping cost. Zero blocks in $\mathbf{W}_k$ deactivate residuals that are irrelevant to the current task. The ESDF term is included separately through J_{obs} in Eq. (17).

State and input box bounds are hard, whereas collision avoidance enters through the soft cost J_{obs} . Consequently, the OCP alone does not provide a hard collision-safety guarantee.

C. Task-Conditioned Reference Generation

The task manager maintains an ordered sequence of base and end-effector tasks and exposes one active task type $\tau \in \{\text{base}, \text{EE}\}$ to the controller. For a base task, the geometric path is planned in $SE(2)$ with the RRT* implementation provided by OMPL [13], [14]. The planner uses the ESDF state-validity callback to reject configurations that violate the configured footprint clearance, and RRT* rewires the sampling tree to reduce path cost. For an end-effector task, the current profile uses OMPL's RRTConnect planner in the bounded Cartesian workspace. After path simplification, let $\gamma_\tau(s)$ denote the resulting geometric path parameterized by progress s . The reference horizon is generated from the measured path progress s_0 as

$$\begin{aligned} s_k &= \text{clip}(s_0 + v_\tau k \Delta t, 0, s_{\text{end}}), \\ \mathbf{r}_k &= \gamma_\tau(s_k), \quad k = 0, \dots, N, \end{aligned} \quad (21)$$

where v_τ is the nominal progress rate. Thus $\mathbf{r}_0$ is the current desired reference, and the remaining samples populate the MPC prediction horizon. A waypoint task is the constant-path special case.

The task type selects the active residual blocks in Eq. (20). A base task activates $SE(2)$ pose and base velocity tracking and

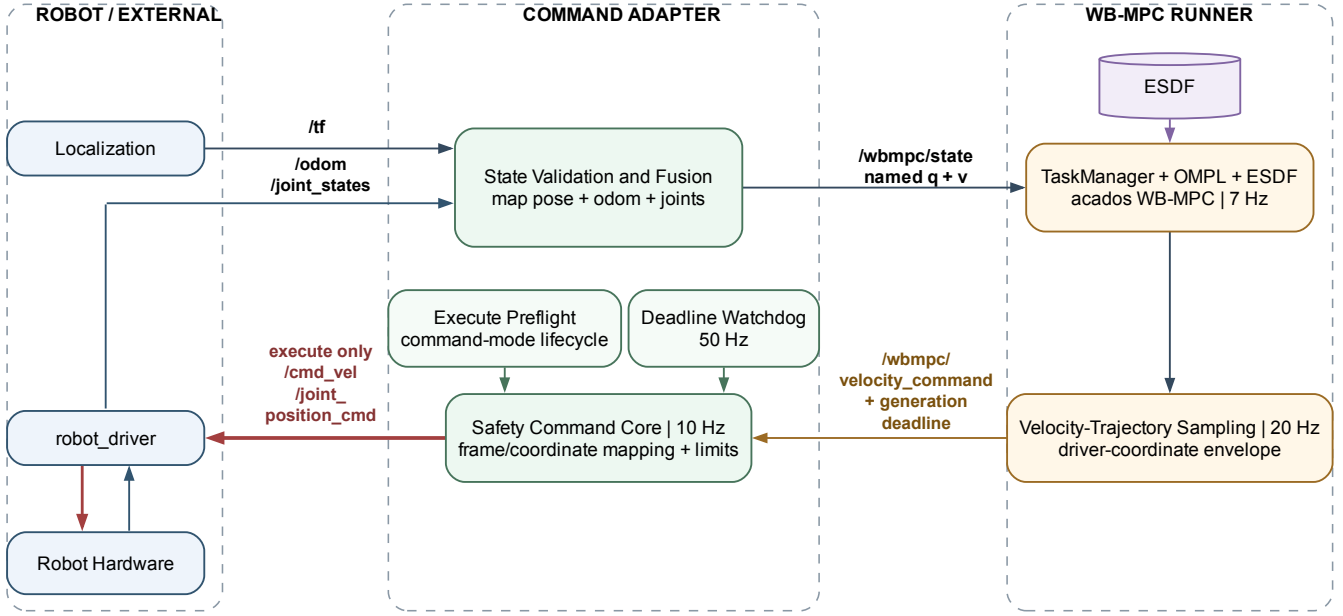


Fig. 2. Platform-abstract ROS 2 nodes and data flow for real-robot execution. The upper path carries feedback into WB-MPC, and the lower path returns the deadline-stamped command envelope. Red hardware outputs exist only with `execute=true`; diagnostic topics and driver-mode service calls are omitted for legibility.

can tighten the preferred arm extension. An end-effector task activates Cartesian position and, when requested, orientation and velocity tracking while allowing the optimizer to coordinate the mobile base and manipulator through whole-body optimal control. Residual masks set irrelevant coordinates to zero weight. Once the active task remains within its configured position and orientation tolerances for the hold period, the manager advances to the next task.

IV. SYSTEM IMPLEMENTATION

This section describes the execution framework and adaptation strategy developed for real-robot deployment.

A. ROS 2 Nodes and Topic Topology

Figure 2 abstracts the real-robot execution path into a driver, a command adapter, and a WB-MPC runner. The `robot_command_adapter` is the only application node connected to the robot driver. It subscribes to odometry, joint feedback, the global-to-base transform, and the driver mode and safety states. Valid temporally aligned base and joint samples are combined into a named `sensor_msgs/JointState` on `/wbmpc/state`. Positions and velocities use the controller ordering, and the header stamp and frame identify the source time and global frame used by the controller.

The `wbmpc_runner` subscribes to this normalized state interface. It owns the task manager, OMPL planners, and WB-MPC instance; the ESDF supplies queries to both planning and control. A worker thread starts planning and optimization at 7 Hz so that state callbacks and the command publisher do not block on a solve. A separate 20-Hz timer samples the most recent valid velocity trajectory,

converts it to the driver’s coordinate representation, and publishes `/wbmpc/velocity_command`. The serialized envelope contains the velocity vector, solver generation, validity flag, source-state stamp, plan origin, and an absolute monotonic validity deadline. The runner and adapter publish JSON status records on `/wbmpc/status` and `/robot_command_adapter/status` for logging and diagnosis.

The adapter consumes the envelope and, in execute mode, publishes base velocity on `/cmd_vel` and a position-oriented joint command on `/joint_position_cmd` at 10 Hz. Consequently, the optimizer never owns a driver topic directly, and all feedback validation, command conversion, deadline enforcement, and driver-mode service calls remain at one ROS 2 boundary.

B. Forward Prediction and Deadline Hold

In simulation, the solver and simulator run sequentially in a blocking loop, so the simulated robot state does not evolve while the optimization is in progress. In real-robot deployment, however, the solver runs concurrently with the continuously moving robot, making compensation for this motion necessary.

The initial state used by the MPC is measured before optimization and does not account for the robot’s motion during computation. Consequently, by the time the optimized command reaches the robot, it is based on an outdated state. The runner can compensate by advancing the measured state over

$$\Delta t_{\text{pred}} = a_{\text{state}} + \hat{t}_{\text{solve}} + t_{\text{act}}, \quad (22)$$

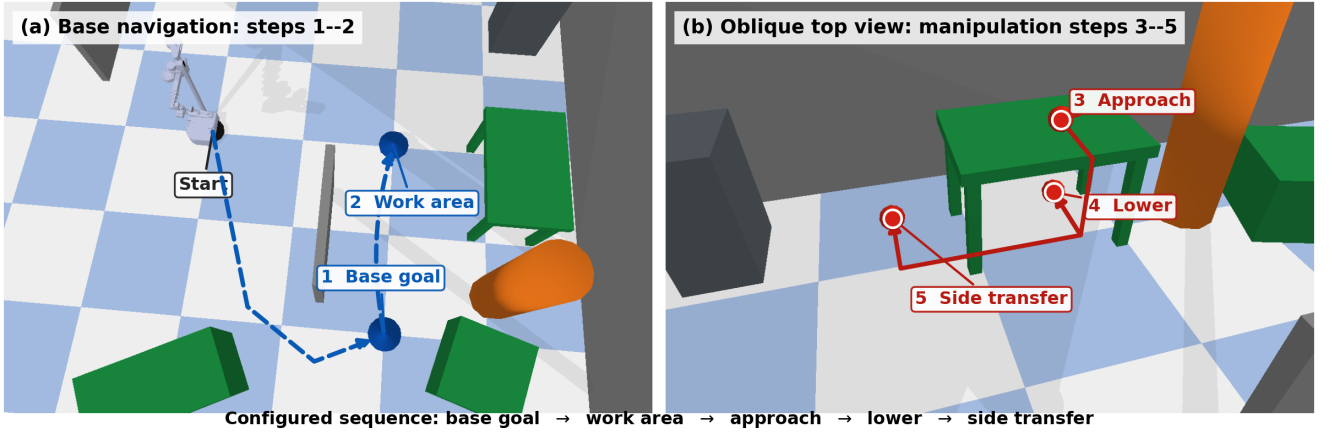


Fig. 3. Complete task specified by the `stretch_esdf_offline_ompl_wbmmpc` profile. (a) The mobile base moves from its initial pose to an intermediate goal and then to the work area. (b) An elevated oblique view, with the robot omitted for clarity, shows the end effector approaching above the work surface, reaching downward, and transferring laterally. The red polylines depict a schematic task-space trajectory that exits the table footprint before changing height or moving laterally; they are not measured paths. Numbering indicates execution order.

where a_{state} is the source age of the synchronized feedback, $\hat{t}_{\text{solve}}$ is an online estimate of planning and optimization time, and t_{act} is the configured dispatch and actuation latency. After each successful cycle, the computation-time estimate is updated as

$$\hat{t}_{\text{solve}}^+ = \text{clip}((1 - \alpha)\hat{t}_{\text{solve}} + \alpha t_{\text{solve}}, t_{\min}, t_{\max}). \quad (23)$$

where t_{solve} is the solve time measured in the current cycle, $\hat{t}_{\text{solve}}$ is its estimate before the update, and the superscript $+$ denotes the updated estimate. The adaptation factor $\alpha \in [0, 1]$ weights the latest measurement, while $t_{\min}$ and $t_{\max}$ bound the updated estimate. The current configuration sets $\alpha = 0.10$, initializes the estimate at 60 ms, clips it to $[20, 120]$ ms, adds 50 ms actuation latency, and rejects a prediction interval above 300 ms.

The initial condition imposed on the MPC is therefore

$$\begin{aligned} \mathbf{x}_0^{\text{MPC}} &= \Phi_{\Delta t_{\text{pred}}}(\mathbf{x}_{\text{meas}}, \mathbf{u}_{\text{last}}), \\ \mathbf{x}_{\text{meas}} &= [\mathbf{q}_{\text{meas}}^{\top} \quad \mathbf{v}_{\text{meas}}^{\top}]^{\top}. \end{aligned} \quad (24)$$

where $\mathbf{u}_{\text{last}}$ is the most recently published acceleration and Φ_{τ} denotes propagation over duration τ using the same discrete robot dynamics and state/input bounds as the OCP. Without forward prediction, $\mathbf{x}_0^{\text{MPC}} = \mathbf{x}_{\text{meas}}$. The controller plans and solves from this initial state and the corresponding future task time.

Command validity is bounded independently of solver termination. The deadline of each optimization cycle also limits the lifetime of the active command. If no replacement is available before this deadline, the runner invalidates the active plan and issues a zero-velocity envelope. A late solution can be admitted under the configured policy, but its time origin and validity interval are reset to its completion time; an expired command interval is never revived.

The hardware adapter independently checks the envelope deadline using a monotonic clock. This check ensures that a timeout produces zero base velocity and an arm position

hold even if the optimization thread stalls. A fresh valid command automatically releases this recoverable hold. In contrast, violations of state integrity, command mapping, device mode, or command ownership trigger a latched stop that requires a restart and new preflight before execution can resume.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup and Metrics

We evaluate whether the same planning and control pipeline can be specialized to mobile manipulators with different dimensions and base kinematics. The framework was evaluated in PyBullet sim environment using both Stretch and mobile-UR10 configurations. The experiment runner, task sequencing, ESDF construction, MPC implementation, and 30-ms simulation step were held fixed across the two platform configurations. Only the robot-specific model, collision geometry, and controller parameters defined by each platform profile were changed. Consequently, the comparison tests cross-platform execution of the framework rather than the relative quality of the two robots or their independently tuned controllers.

The evaluated specializations are summarized in Table I. Stretch combines an eight-coordinate model with the nonholonomic constraint in (5); the mobile UR10 uses a nine-coordinate model and an omnidirectional base. Their collision models also differ substantially, with 34 and 93 spheres, respectively. These differences exercise distinct dynamics and geometry branches while preserving the surrounding software and experimental protocol.

Figure 3 illustrates the complete composite task in the Stretch profile. The robot first visits two base goals to enter the work area, then executes an approach, vertical reach, and lateral transfer with the end effector. Separating the navigation and manipulation views keeps all five configured targets visible while preserving their spatial relationship to the same simulated workspace.

TABLE I
EVALUATED SPECIALIZATIONS OF THE SHARED FRAMEWORK

Property	Stretch	Mobile UR10
Optimized coordinates	8	9
Arm coordinates n_a	5	6
Base model	nonholonomic	holonomic
State dimension n_x	16	18
Collision spheres	34	93
Extra ESDF distance	0.07 m	0.10 m

TABLE II
SIMULATION RESULTS FOR TWO ROBOT CONFIGURATIONS

Metric	Stretch	Mobile UR10
Base position RMSE	20.5 mm	14.1 mm
Base yaw RMSE	6.55 mrad	3.31 mrad
EE position RMSE	16.6 mm	18.5 mm
Minimum surface clearance	73.2 mm	101.5 mm
Minimum safety margin	3.22 mm	1.47 mm
Controller mean / P95	81.4 / 141.0 ms	71.4 / 80.4 ms
Controller maximum	785.3 ms	799.3 ms
Calls above 120 ms	7.50%	1.05%
Solver status 0 / 2	1296 / 38	1998 / 2

Tracking is evaluated only over samples for which the corresponding reference is active. For a position sequence $\mathbf{p}_k$ with active-reference index set $\mathcal{I}$, we report

$$e_{\text{RMSE}} = \sqrt{\frac{1}{|\mathcal{I}|} \sum_{k \in \mathcal{I}} \|\mathbf{p}_k - \mathbf{p}_k^{\text{ref}}\|_2^2}. \quad (25)$$

Base-yaw RMSE is defined analogously after wrapping each angular error to $[-\pi, \pi]$. The final task-target error measures distance to the terminal goal, and is therefore distinct from error to the final sample of a time-varying reference. Collision performance is characterized by the minimum surface clearance, $d - r$, over all queried sphere centers, and by the minimum safety margin, $d - r - d_{\text{safe}}$.

Runtime measurements enclose the complete controller call, including parameter construction, ESDF linearization, optimization, and diagnostics. We additionally report solver outcomes, fallback activations, and the fraction of calls exceeding 120 ms. All timing results were obtained on an Intel Core i9-14900HX.

B. Cross-Platform Tracking

Across both configurations, the shared controller maintained centimeter-level tracking despite differences in base mobility, arm dimension, collision geometry, and tool-frame definition. Base-position RMSE ranged from 14.1 to 20.5 mm, base-yaw RMSE from 3.31 to 6.55 mrad, and EE-position RMSE from 16.6 to 18.5 mm. These results indicate that the same optimization and execution pipeline can accommodate distinct kinematic structures without embedding a single platform’s motion model in the common planner. Platform-specific constraints, such as the nonholonomic dynamics in Equation (5), enter through the configured model.

The comparison is intended as a portability test rather than a ranking of the two robots. Each configuration instantiates its own state dimension, base dynamics, arm coordinates, collision representation, and tracking weights, while retaining the same controller structure and execution path. Variations in completion and terminal error therefore reflect the combined effects of platform dynamics, scenario duration, reference evolution, and tuning; they should not be attributed to morphology alone. The aggregate results are summarized in Table II.

Similar aggregate RMSE values can nevertheless mask platform-dependent error structure. Axis-wise EE errors differ across the two configurations, with the largest contribution occurring on different axes and with different temporal profiles. For this reason, cross-platform evaluation should report both aggregate tracking metrics and time-resolved errors. Figure 4 provides these complementary views.

C. ESDF Clearance

All ESDF queries were valid on both robots. Minimum executed-state surface clearance, defined as distance minus sphere radius, was 73.2 mm for Stretch and 101.5 mm for the mobile UR10. After subtracting their separately configured 70- and 100-mm safety distances, the minimum executed-state margins were positive but small: 3.22 and 1.47 mm. The pre-optimization warm-start horizon reached -24.5 and -83.9 mm, respectively. These negative warm-start margins were not executed-state collisions. During re-optimization, the ESDF collision cost further safeguarded execution by steering the candidate trajectory away from obstacles. If acados terminated at its SQP-iteration limit (status 2), a separate post-solve acceptance check examined the resulting trajectory before it could be issued: all state and control entries had to be finite, every ESDF query for every collision sphere and shooting node had to be valid, and the minimum horizon margin, $d - r - d_{\text{safe}}$, had to be no smaller than the configured 0-mm threshold. Failure of any check caused a zero-velocity fallback. Thus, the negative-margin warm starts above were not issued as commands. The same post-solve horizon margins should in future be logged for every solver outcome, rather than only when status 2 triggers this check.

D. Solver and Runtime Timing

Complete controller time averaged 81.4 ms for Stretch and 71.4 ms for the mobile UR10. Their P95 values were 141.0 and 80.4 ms, but isolated maxima of 785.3 and 799.3 ms show long tails on both. The solver-status row in Table II uses the acados return codes: status 0 denotes successful convergence, whereas status 2 denotes the configured SQP-iteration limit. The status-2 acceptance check defined above admitted all 38 such Stretch cycles; of the mobile UR10’s two status-2 cycles, one was admitted and one produced the zero-velocity fallback. The 93-sphere UR10 profile spent more mean time in parameter update and ESDF linearization (51.3 and 37.4 ms) than Stretch (30.6 and 15.9 ms), yet its easier OCP solves averaged 8.74 rather than 41.6 ms. This decomposition shows that robot geometry

Time-resolved end-effector tracking error

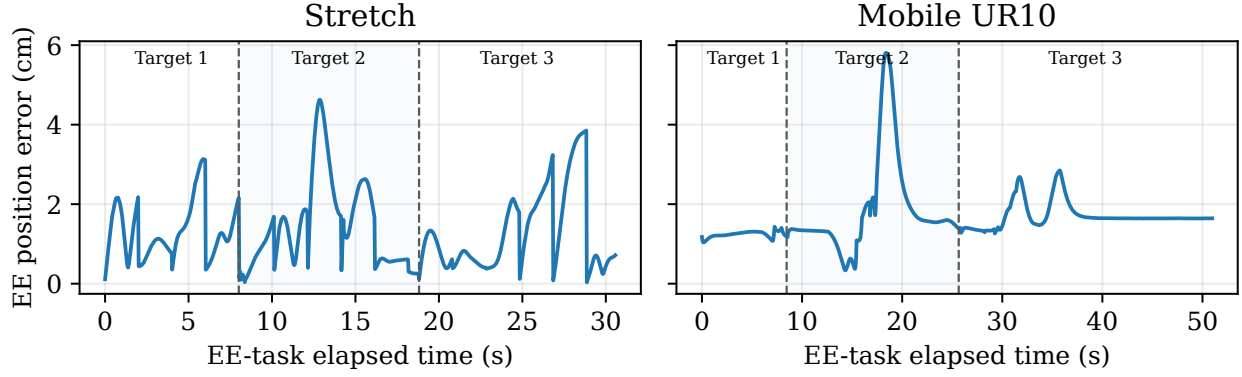


Fig. 4. Euclidean EE reference-error histories for both configurations. Time is measured from the first active EE-reference sample in each evaluation, and both panels use the same vertical scale. Dashed lines mark transitions between T1 (Approach), T2 (Reach Low), and T3 (Reach Side).

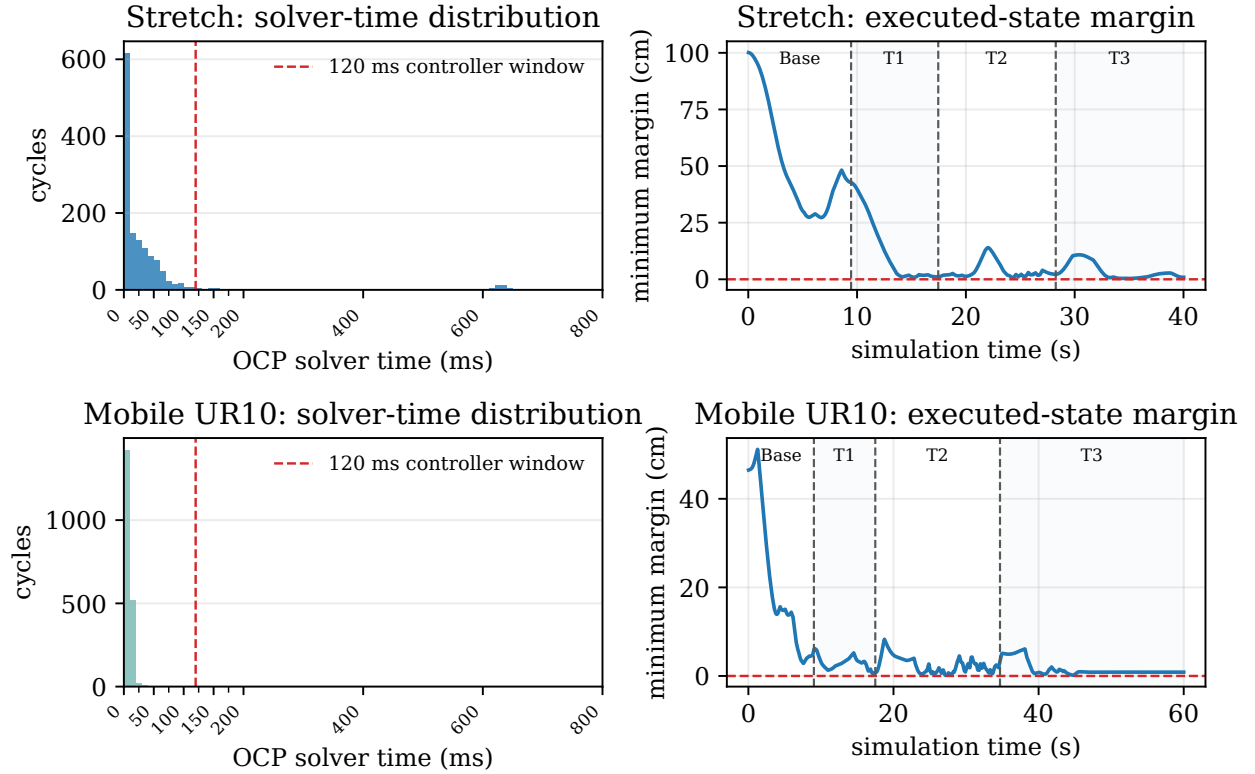


Fig. 5. OCP solver-time distributions and minimum executed-state ESDF margins for Stretch (top) and mobile UR10 (bottom). Solver-time histograms use 10-ms bins, with finer ticks over 0–200 ms. In the margin panels, vertical dashed lines separate the base-motion phase from T1 (Approach), T2 (Reach Low), and T3 (Reach Side). The 120-ms line shows the shared controller validity-window reference; the zero-margin line denotes each profile’s own configured collision boundary.

and optimization difficulty affect different runtime components; configurability does not imply platform-independent timing.

Table III gives the distribution and solver-iteration details. Parameter update includes all shooting-node parameters, and ESDF linearization is nested within that work; the rows must therefore not be summed as independent totals. The UR10’s

larger sphere set increases the largely deterministic parameter cost, whereas Stretch’s difficult SQP cycles dominate its upper tail. In particular, the median-to-P99 jump from 53.6 to 700.6 ms for Stretch is much larger than the mobile UR10’s 68.6 to 120.3 ms.

Figure 5 shows why both central tendency and tails are

TABLE III
RUNTIME AND SOLVER BREAKDOWN

Metric	Stretch	Mobile UR10
Controller median / P99 (ms)	53.6 / 700.6	68.6 / 120.3
OCP solve mean / P95 (ms)	41.6 / 100.2	8.74 / 16.0
Parameter update mean (ms)	30.6	51.3
ESDF linearization mean (ms)	15.9	37.4
SQP iterations mean / P95	32.0 / 77.7	5.15 / 10.0
SQP iterations maximum	500	500
Accepted nonzero status	38	1
Zero-velocity fallback	0	1

necessary. It compares the OCP solver-time distribution with the executed-state collision-margin history for each robot. The margin histories are divided into the base-motion phase and three end-effector task phases. No causal relationship should be inferred directly from this plot: solver difficulty depends on tracking, active costs, bounds, and local ESDF geometry simultaneously.

E. Real-Robot Deployment

The control framework was successfully deployed on the physical Stretch platform. Spectacular AI SLAM localizes the base in the shared map frame. The environment is reconstructed with nvblox, and its ESDF is exported offline and loaded by the controller [10]. Robot state, Spectacular AI localization, and static ESDF queries are then combined to optimize and execute whole-body commands through the Stretch interface, demonstrating the complete mapping-localization-control pipeline on real hardware. Videos are available in the online demo folder.

VI. DISCUSSION

The two evaluated configurations provide executable evidence that the central planning and control framework is not specific to Stretch. Without changing the common experiment runner or MPC class, it instantiated different state dimensions, arm chains, tool frames, collision-sphere sets, clearances, and base dynamics. Both configurations tracked moving references to centimeter accuracy and used base motion during manipulation. This supports adaptability to the two tested mobile manipulators.

The framework is effective in part because planning and control provide two complementary layers of collision avoidance. First, OMPL uses ESDF-based state validity checks to generate a collision-checked reference path with the configured clearance. This supplies the controller with a globally meaningful route around obstacles instead of requiring a short MPC horizon to discover one. Second, WB-MPC evaluates the ESDF collision cost over the predicted whole-body trajectory and can modify the local motion while enforcing the robot dynamics and tracking the reference. The planner therefore protects the geometric path, while the MPC provides additional protection against clearance loss during execution. Because both layers query the same ESDF, their collision reasoning remains geometrically consistent. The positive executed-state margins

observed for both robots support the effectiveness of this two-layer design in the evaluated scenarios.

The runtime decomposition also highlights the role of base mobility in OCP difficulty. Although the mobile UR10 uses the larger collision-sphere set and spends more time on parameter updates and ESDF linearization, its OCP solves required only 8.74 ms and 5.15 SQP iterations on average, compared with 41.6 ms and 32.0 iterations for Stretch. The UR10’s holonomic base permits independent planar translation, whereas Stretch’s nonholonomic constraint couples translational motion to its heading. The resulting increase in instantaneously feasible correction directions can make simultaneous tracking and collision avoidance easier to solve. The results therefore suggest that base mobility can affect optimization difficulty more strongly than the collision-model size alone.

The positive 3.22- and 1.47-mm minimum margins show that both robots respected their configured collision boundaries throughout the evaluated trajectories. Together with the tracking results, this demonstrates simultaneous task execution and modeled whole-body clearance for two distinct mobile-manipulator configurations. The real Stretch deployment uses a static ESDF exported from nvblox and aligned with Spectacular AI SLAM, providing a consistent geometric reference across mapping, planning, and control. As expected for a precomputed map, changes made after export are outside its representation; online updates and platform safety mechanisms can complement this map when operating in dynamic environments.

VII. CONCLUSION

This work demonstrates that heterogeneous mobile manipulators can share a configurable ESDF-aware planning and whole-body control pipeline. OMPL provides a globally collision-checked reference, while the WB-MPC collision cost adds local clearance protection during execution. The same implementation achieved centimeter-level tracking and positive executed-state margins on nonholonomic Stretch and holonomic mobile-UR10 models. The UR10 OCP was easier to solve despite its larger collision model, showing the importance of base mobility to optimization difficulty. Deployment on the physical Stretch with Spectacular AI SLAM, an offline nvblox ESDF, and the guarded ROS 2 interface confirms that the architecture extends beyond simulation.

VIII. FUTURE WORK

Future work will first investigate an object-centric ESDF constructed from semantic object detections, estimated poses, and geometric models. Compared with an ESDF built directly from RGB-D measurements, this representation could provide more stable collision boundaries in the presence of depth noise and missing measurements. A second direction is to close the loop between perception, mapping, navigation, and manipulation. A shared scene representation would allow new observations to update the map, navigation to position the mobile base, and WB-MPC to complete manipulation while triggering replanning when the scene changes.

APPENDIX

Code repository:

https://github.com/stateirving/mobile_manipulation

Demonstration videos:

[https://drive.google.com/drive/folders/](https://drive.google.com/drive/folders/1bsqAc2rYdHWyNgyATlZ9J_RCnpOuP5xE?usp=drive_link)

[1bsqAc2rYdHWyNgyATlZ9J_RCnpOuP5xE?usp=drive_link](https://drive.google.com/drive/folders/1bsqAc2rYdHWyNgyATlZ9J_RCnpOuP5xE?usp=drive_link)

REFERENCES

- [1] C. C. Kemp, A. Edsinger, H. M. Clever, and B. Matulevich, “The design of stretch: A compact, lightweight mobile manipulator for indoor human environments,” in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3150–3157.
- [2] H. Seraji, “A unified approach to motion control of mobile manipulators,” *The International Journal of Robotics Research*, vol. 17, no. 2, pp. 107–118, 1998.
- [3] A. De Luca, G. Oriolo, and P. Robuffo Giordano, “Kinematic control of nonholonomic mobile manipulators in the presence of steering wheels,” in *2010 IEEE International Conference on Robotics and Automation*, 2010, pp. 1792–1798.
- [4] M. Bhardwaj, B. Sundaralingam, A. Mousavian, N. D. Ratliff, D. Fox, F. Ramos, and B. Boots, “STORM: An integrated framework for fast joint-space model-predictive control for reactive manipulation,” in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 750–759. [Online]. Available: <https://proceedings.mlr.press/v164/bhardwaj22a.html>
- [5] C. Pezzato, C. Salmi, E. Trevisan, M. Spahn, J. Alonso-Mora, and C. H. Corbato, “Sampling-based model predictive control leveraging parallelizable physics simulations,” *IEEE Robotics and Automation Letters*, vol. 10, no. 3, pp. 2750–2757, 2025.
- [6] V. Vasilopoulos, S. Garg, P. Piacenza, J. Huh, and V. Isler, “RAMP: Hierarchical reactive motion planning for manipulation tasks using implicit signed distance functions,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 10 551–10 558.
- [7] M. V. Minniti, F. Farshidian, R. Grandia, and M. Hutter, “Whole-body MPC for a dynamically stable mobile manipulator,” *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3687–3694, 2019.
- [8] J. Pan, S. Chitta, and D. Manocha, “FCL: A general purpose library for collision and proximity queries,” in *2012 IEEE International Conference on Robotics and Automation*, 2012, pp. 3859–3866.
- [9] H. Oleynikova, Z. Taylor, M. Fehr, J. Nieto, and R. Siegwart, “Voxblox: Incremental 3d euclidean signed distance fields for on-board mav planning,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 1366–1373.
- [10] A. Millane, H. Oleynikova, E. Wirbel, R. Steiner, V. Ramasamy, D. Tingdahl, and R. Siegwart, “nvblox: GPU-accelerated incremental signed distance field mapping,” *arXiv preprint arXiv:2311.00626*, 2023.
- [11] M. Gaertner, M. Bjelonic, F. Farshidian, and M. Hutter, “Collision-free MPC for legged robots in static and dynamic scenes,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 8266–8272.
- [12] J.-R. Chiu, J.-P. Sleiman, M. Mittal, F. Farshidian, and M. Hutter, “A collision-free MPC for whole-body dynamic locomotion and manipulation,” in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 4686–4693.
- [13] I. A. Şucan, M. Moll, and L. E. Kavraki, “The open motion planning library,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.
- [14] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.